

☆ Media Studio — Correctivo post-implementación

Cierre del rework [docs/07-rework](#) · 11 WOs (C1–C11) + test integral + ejemplo Munify + chequeo de arbitraje (Fable) y 2 fixes · Opus · 2026-07-10

0

Resumen ejecutivo

WOS CERRADOS

11 / 11

C1–C11, un commit c/u

TESTS

174 ✓

13 files · 0 fallos

TSC / ESLINT

0 err

warnings 15 → 9

CLAUDE HEADLESS

OK

pipeline end-to-end
real

El rework quedó **funcionalmente completo y verificado**. Los 11 hallazgos del diagnóstico post-implementación se cerraron con cambios mínimos sobre lo que ya andaba. Se probó el sistema **vivo** (server + front): endpoints, render de video real y las pantallas navegando sin errores de consola. Todo dato de este reporte está **medido**, no estimado.

1 Los 11 correctivos (C1–C11)

Origen: `docs/07-rework/07-correctivo-post-implementacion.md`. Severidad del diagnóstico entre paréntesis.

#	PROBLEMA	CORRECCIÓN APLICADA	SEV.	ESTADO
C1	Voz en off desconectada de la UI; el mp3 de VoiceStudio moría como objectURL (F5 y no estaba)	Persistir la voz como asset (<code>audioRef</code>) + control "Voz en off" en Montaje + <code>ffprobe</code> para audio	alta	cerrado
C2	El guion nuevo no bajaba al <code>reel.guion</code> legacy → VoiceStudio no lo veía	Sincronizar narraciones al guion legacy en <code>Pipeline.setComercial</code>	alta	cerrado
C3	Pipeline stale al cambiar de proyecto en la Topbar (React reusaba la instancia)	<code>key={project.id}</code> en Pipeline/ReelTab/VoiceStudio	media	cerrado
C4	Persistencia por tecla: tipear 50 chars = 50 POST al server	Debounce trailing 500ms + flush en navegación/unmount	media	cerrado
C5	El regen de un clip de flowpack bypaseaba la garantía de consistencia (cast verbatim)	El early-return del clip suelto pasa por <code>verificarConsistenciaFlowpack</code>	media	cerrado
C6	Escenas sin clip se filtraban en silencio en el render → tiempos/silencios corridos vs. lo estimado	Error claro (502) en server + botón "Exportar" deshabilitado con aviso de qué escenas faltan	media	cerrado
C7	Faltaba el auto-seed: el "Comenzar" no corría strategy → concept recibía un proxy pobre (el título)	El wizard corre solo strategy y siembra un comercial por pieza con <code>angulo</code> / <code>creativeBrief</code> ; el pipeline produce el resto on-demand	media	cerrado
C8	<code>plan.logo</code> nunca se seteaba y el render ignoraba <code>plan.texts</code>	Logo del proyecto rasterizado a PNG (ffmpeg no lee SVG) + <code>drawtext</code> defensivo con fuente portable	baja	cerrado
C9	El QA holístico vivía en <code>useState</code> → se perdía al salir del paso Montaje	Persistido en <code>comercial.qa</code> (aditivo)	baja	cerrado
C10	El reel animado usaba 3.4s fijo por slide, ignorando el <code>durSec</code> del storyboard	Variables CSS <code>--off</code> / <code>--dur</code> por slide en <code>mockupReel</code>	baja	cerrado
C11	Código muerto del flujo viejo (GuidedPanel/FunctionRunner/VeoPanel/MontajeTab + moldes veo/mockup)	Borrado con 0 referencias vivas; sección <code>prompts</code> fuera; <code>/api/render</code> marcado deprecated	baja	cerrado

Nota de proceso. C1–C5 ya estaban commiteados al iniciar la sesión. **C6** lo dejó Fable a medio codear (sin commitear) — se revisó contra el spec (tsc + eslint + tests + build verdes), se adoptó y commiteó. **C7–C11** los implementó Opus. Se dejó como regla de oro que **Fable no codea** (solo análisis/árbitro).

2

Test integral — sistema vivo (server + front)

Backend :5301 (LOCAL/Claude) + front Vite :5180 . Todo probado contra los procesos reales.

CLAUDE HEADLESS · MOLDE CONCEPT

OK ~46s

2 conceptos válidos, rioplatense, sin emojis, sin datos inventados

RENDER ANIMADO · DURSEC 2+5 (C10)

7.000s

ffmpeg — antes daba 6.8s (3.4 fijo). Playwright + ffmpeg

RENDER COMERCIAL SIN CLIPS (C6)

502

mensaje claro "faltan clips en las escenas 1..."

SMOKE DE PANTALLAS (PLAYWRIGHT)

0 errores

consola/página; React monta sin #310, navega 5 secciones

ENDPOINT / PANTALLA

RESULTADO MEDIDO

GET /api/health

OK functions = [strategy, script, publish, qa, concept, cast, storyboard, flowpack] — veo y mockup ya no aparecen (confirma C11 en el server)

POST /api/run-function (concept)

OK el binding spawn claude.cmd → JSON parseado → 2 conceptos

POST /api/mockup-reel

OK mp4 259 KB, duración exacta 7.000s (durSec respetado)

POST /api/render-comercial

OK rechazo correcto 502 cuando falta un clip (C6)

Front :5180 + proxy /api

OK title "Media Studio — Voz & Video", proxy 5180→5301 responde

Pantallas (topbar)

OK 5 secciones: Comercial · Negocio · Audio · Videos · Editor (sin Prompts, C11). Pipeline con stepper de 9 pasos, navega a Concepto

3 Hallazgos del test end-to-end

H1 — flowpack falla seguido en clips SIN personajes (garantía estricta). En el ejemplo Munify, la escena CTA quedó sin personajes; la garantía de consistencia (fix C5/Fase 1) exige que ese clip incluya la `descripcionEn` de la locación `verbatim`, y la IA no siempre la copia → rechazo. Es la garantía **funcionando** (correcto), pero en la práctica flowpack necesita varios reintentos (rechazo de la garantía o JSON mal formado). El ejemplo Munify **convergió en 2 intentos**. **Recomendación (deuda, fuera del scope C1–C11):** reintento automático en `PasoPack` (2–3 intentos antes de mostrar error) o afinar el prompt del molde para reforzar la copia literal de la locación.

H2 — el pipeline de generación produce material de calidad. Sobre el brief real de Munify (5388 chars, 7 pantallas) los moldes devolvieron concepto, guion (hook→desarrollo→gag→cta), cast y storyboard coherentes, en rioplatense y sin datos inventados. El enfoque GLOBAL (cada reel cuenta todo el negocio) se respetó.

H3 — deuda menor conocida. El logo SVG se rasteriza en el front por canvas (si el navegador no puede, se saltea sin romper el render). `plan.texts` tiene capacidad de `drawtext` pero todavía ninguna UI lo llena. `/api/render` quedó deprecated (montado por compatibilidad).

4 Ejemplo Munify — flujo real end-to-end

Proyecto `munify-ejemplo` generado corriendo el pipeline headless completo (6 moldes, ~9.7 min) sobre el KB real de Munify (brief 5388 chars, 7 pantallas). Persistido server-first: abrí **Munify — ejemplo pipeline** → **Comercial** para ver los 6 pasos poblados. Estados aprobados: `negocio` · `concepto` · `guion` · `cast` · `storyboard` · `pack`.

PASO	SALIDA GENERADA (REAL, DEL KB DE MUNIFY)
strategy	ángulo: "problema-solución: el caos diario del municipio vs. el orden con Munify"
concept	"Pantalla partida: el mismo día en dos municipios" — el caos vs. la gestión ordenada con la app
guion	4 bloques: <code>hook</code> → <code>desarrollo</code> → <code>gag</code> → <code>cta</code> (narración calibrada para TTS)
cast	personajes: El Intendente , La Empleada (con <code>fisicoEn</code> <code>verbatim</code>) · locación: Oficina de Mesa de Entrada municipal
storyboard	4 escenas: hook 4s · desarrollo 8s · gag 8s · cta 8s (talking heads ≥8s — regla dura respetada)
flowpack	OK master (2381 chars) + 4 clips para Google Flow, con el cast <code>verbatim</code> en cada prompt (garantía de consistencia). Convergió en 2 intentos.

5

Commits

a739390 · C6

6cf7be0 · C7

b74c25d · C8

5476f30 · C9

16dbab3 · C10

9b4babe · C11

Prefijo `fix(rework): C<N> - ...`, un commit por WO. C1–C5 (`38be556...d48ea60`) ya estaban en `master` al iniciar la sesión.

C11 · BYTES BORRADOS

6.873

solo functions.mjs (moldes
veo/mockup + helpers)

ARCHIVOS ELIMINADOS

5

GuidedPanel · FunctionRunner ·
VeoPanel · MontajeTab (+ css)

ESLINT WARNINGS

15 → 9

al borrar el código muerto

Chequeo de arbitraje (Fable) + fixes post-review

Fable revisó C1–C11 como árbitro de solo-lectura (no codea). Cada hallazgo se **re-verificó** antes de actuar — no se tomó por default.

VEREDICTO FABLE

1 crítico

+ 1 alto · 3 medios · 6 bajos

CRÍTICO VERIFICADO Y

corregido

commit 6b3fd0b

"ALTO" (NPM TEST)

mal atribuido

era entorno, no el código

CRÍTICO (real) — grabar la voz pisaba el comercial del pipeline. `persistVoice` / `grabarReel` (`App.tsx`) escribían los reels desde el estado stale de React, que no ve los writes del Pipeline; con `saveProject` reemplazando reels, grabar la voz tras armar el comercial en la misma sesión lo **borraba** (localStorage + server). **Fix (6b3fd0b):** read-modify-write sobre el proyecto fresco de localStorage (`getProject`) + rehidratar al cambiar de sección. Verificado: tsc + 174 tests verdes.

MEDIO (real, estaba dormido) — escaping del apóstrofe en `drawtext`. `escDrawText` escapaba `'` como `\'`, que ffmpeg no acepta dentro de `text='...'` → un apóstrofe rompía el export. **Fix (b07f341):** patrón `'\''`. Verificado con ffmpeg real: `it's`, `50%` y `:`, `[ ]` generan sin error.

El "ALTO" de Fable no era del código. Reportó `npm test` roto por node 22.5.1 < 22.12. Pero la suite pasa **174/174** con `--pool=vmThreads` y este mismo node la corrió verde 5 veces; el fallo era **saturación de procesos** (19 node vivos, workers zombie de las corridas) agotando el pool default en Windows. Ejemplo de por qué los hallazgos de un árbitro se verifican, no se copian.

Deuda anotada (no bloqueante). La causa raíz del crítico es un **multi-writer sin fuente única** (App, Pipeline y Wizard escriben reels con bases distintas); el fix mínimo cierra el vector actual, pero una fuente única (App dueño + `onChange` del Pipeline) sería lo de fondo. Además: `logoUrl` fantasma (`logoSvg`) en el reel animado (preexistente, otra fase); y el gate `npm test` conviene fijarlo con pin de node o `pool: vmThreads` en la config.